TAG COVERAGE — does the engine actually use what the artifact declares?

Version 3.40.0 · 2026-08-04

> *Will, 2026-08-04: "CAN WE DESIGN TESTS THAT CHECK ALL THE MOST COMMON MOVES, ITEMS, ABILITIES, AND MONS AND SEE IF ALL THE TAGS ACTUALLY GET USED IN THE ENGINE" — and, on why he should not have to supply the list himself: "I CANNOT TELL YOU ALL THE EXAMPLES".*

He should not have to, and this document is the machine that means he does not.

---

1. Why this class of bug is structural rather than accidental

`engine/tag_dex.js` derives **174 distinct tags** from Showdown's own data. `engine/medicham2-browser.js` then hand-writes a consumer for each one. So every tag needs a person to remember to wire it, and **nothing fails when nobody does.**

That is the **Expression Problem** (Wadler, 1998): a system can be arranged so new *data* is cheap to add or new *operations* are, but not both for free. Showdown takes the object-oriented direction — behaviour lives on the move, as `onTryHit` / `onModifyDamage` handlers — so a new move is automatically live and there is no wiring step to forget. ABRA takes the other direction: tags are data, consumers are separate code. That is a legitimate choice and it is the one functional languages make, **but it is only safe with an exhaustiveness check**, and the check was never written.

The tighter name for the coupling is **connascence of meaning** (Page-Jones): producer and consumer must agree on what `extendsDuration` *means*, with nothing enforcing the agreement, across the longest distance in the codebase.

Evidence that this is the generator rather than a one-off: WIRE 71. `extendsDuration` had four consuming routes and **three wrote a literal 5 for months.** Every test passed.

---

2. Two tiers, and neither is sufficient alone

Tier 1 — CONSUMPTION. `tests/test-tag-consumed.js`. Built and running.

`engine/tags.js` has counted tag reads since the day it was written and **nothing had ever called** `hits()`. Worse, it could not have answered this question: `param()` counted a tag only when the entity *carried* it, and `has()` counted nothing at all — so a zero reading was ambiguous between two opposite diagnoses. Split into **ASKED** (any lookup) and **FOUND** (the entity carried it), combined with a static scan for the tag name in engine source, the reading resolves into four states:

static	runtime	verdict	what it means
not named	ASKED = 0	**DEAD**	no line of engine code looks for this tag
named	ASKED = 0	**UNREACHED**	a consumer exists; the sweep never ran that path — a gap in the test
—	ASKED > 0, FOUND = 0	**STAGED**	consumer ran; no carrier was supplied
—	FOUND > 0	**LIVE**	a consumer read real parameters off a real entity

Each instrument covers the other's blind spot. The runtime counter cannot prove a consumer is absent, only that a path was not reached. The static scan cannot see a tag name built at runtime — `withTag` takes the name as an argument. **DEAD requires both to agree**, which is as close to decisive as this pair can get.

`FOUND > 0` **IS NECESSARY AND NOT SUFFICIENT.** A consumer can read a tag and ignore its payload. `spreadAll` is read by tag NAME to build a set while its `hitsAlly` param is never consulted; a generic burn would satisfy `inflictsBurn` while modelling the wrong mechanic. That is what Tier 2 is for.

Tier 2 — MUTATION. Specified here, not yet built. ENGINE owns it.

Mutation testing (DeMillo, Lipton & Sayward, 1978): break the code deliberately and check a test goes red. If deleting the Water Absorb branch leaves every test green, the suite does not test Water Absorb.

The standard objection is cost — O(mutants × suite), which is why people skip it. **Our case is unusually favourable: the tag list supplies the mutation operators for free.** We do not need random mutants. Remove tag *T* from the artifact in memory, re-run a fixed battery under a fixed seed, and compare a digest of the whole turn-by-turn state:

- **state differs** → the read has an effect. Genuinely LIVE.
- **state identical** → the tag is read and ignored, *or* the battery cannot express it. Distinguish those exactly as `tests/test-interaction-matrix.js` already does — by asking whether the **reference engine's** two arms differ. If Showdown does not care either, the case is INERT and scoring it would count a pass no engine could fail.

Implementation note, so it is not rediscovered: `engine/tags.js` memoises the artifact into `DB` on first `load()`. Mutation needs an injection point — add `__setDB(obj)` rather than clearing the require cache, which would also drop the counters. **And `__setDB` alone is not enough (found 2026-08-05):** `medicham2-browser.js` **builds its tag-derived sets —** `SPREAD`**,** `HITS_ALLY`**, the terrain table, the priority-block map — at module load, so an injected DB silently no-ops for every set-building tag and scores it read-and-ignored, the false-DEAD direction.** `__setDB` **ships with a derived-set rebuild hook, and the harness gate includes one set-building tag to prove the hook fires. Operators are per-tag AND per-param — tag removal cannot see a read-and-ignored param, which is the WIRE 71 shape and the** `hitsAlly` **case this section opens with. Run a small seed battery per mutant: one seed can miss a probabilistic effect, and PRNG-stream shift can move the digest for unrelated reasons; both error directions get named in the artifact.**

An `ABRA_TAGS_OFF=1` switch already exists and blanks **every** lookup. That is the all-or-nothing control for "did wiring the artifact help at all". Tier 2 is the per-tag version of it.

---

3. What Tier 1 found on its first honest run

The first run was wrong and the failure is recorded because it is instructive. Classifying every `ASKED = 0` tag as DEAD reported **132**, including `flinches`, `redirects`, `reversesSpeed` (Trick Room), `passiveHeal` (Leftovers) and `halvesDamage` (screens) — all demonstrably wired. The sweep drove the damage and switch-in paths and never entered the **battle loop**, where those are consumed. It was measuring its own path coverage and reporting it as an engine defect: a **false DEAD**, which is the dangerous direction, because it sends someone to wire a mechanic that already works.

With both instruments combined, over 174 tags:

	count
LIVE	33
STAGED (sweep supplied no carrier)	4

	count
UNREACHED (named in source, path not driven)	77
DEAD (no literal anywhere, never asked)	**61**

The 61 are not 61 missing mechanics. They are mostly a SECOND RULEBOOK.

Split by what kind of thing carries them:

kind	tags	corpus uses
move	35	468,556
ability	19	58,238
item	4	36,092
mixed	3	10,154

The move tags are largely **duplicated facts**. `CHOMP/data/move-effects.json` is a second rulebook of **954 moves** carrying `status`, `secondary`, `recoil`, `drain`, `weather`, `terrain`, `priority`, `heal`, `multihit`, `selfBoostsAlways`, `targetBoostsAlways` — and **the engine reads that one**. Flinch does not come from the `flinches` tag; it comes from `MOVE_EFFECTS`. So the mechanic works and the tag is dead weight.

That is `CLAUDE.md`'s own rule broken at scale. *"Two files that both decide Choice Scarf multiplies Speed by 1.5 will disagree eventually, and the disagreement will be invisible because both keep working."* Here it is two files deciding which moves flinch, which moves recoil, and which moves set weather — and tonight's WIRE 71 was exactly a weather fact that had drifted between the two.

The sharp residue: 26 ability and item tags, 104,484 uses, that cannot be covered by move-effects.json

`move-effects.json` describes moves only. An ability or item tag with no consumer has no second rulebook to fall back on:

tag	uses
`megaStone`	29,790
`damageBoost`	12,085
`onSwitchInDrop`	10,415
`boostsWhenLowered`	7,965
`priorityMod`	7,958
`contactPunish`	6,829
`speedMult`	6,141
`stabBoost`	4,468
`speedCond`	3,564
`blocksStatusMoves`	2,539
`accuracyMod`	2,537

These are candidates, not confirmed gaps, and the distinction matters. Several are implemented by **hardcoded name** rather than from the artifact — `intimidate` appears 5 times in the simulator and `prankster` 10, as literal names. So the mechanic fires today and **the engine will silently fail to pick up any new ability of the same shape**, which is precisely what `CLAUDE.md` means by *"match on tag shape, never on a name, so an ability added later is picked up without editing the engine."*

Two tags also appear to describe one mechanic — `contactPunish` (dead, 6,829 uses) beside `punishesAttacker` (live). One of those is redundant and nothing has noticed.

---

4. The work, in order

(2026-08-05: step 1 is DONE — the full 26-way triage with per-tag verdicts is in docs/ENGINE.md under "THE LAYER 0 PASS", wires 90–112. DEAD fell 61 → 40. The `contactPunish` / `punishesAttacker` *redundancy resolved in punishesAttacker's favour;* `blocksStatusMoves` *was redundant AND over-matched (Telepathy, Wonder Guard); the* `onSwitchInDrop` *enrichment caught Download as a §4-style over-match.* `TAGS.__setDB` *+ a rebuild-hook registry landed in engine/tags.js as step 3's injection point, and* `tests/probe_red_demo.js` *is the first user of the mutation operation.)*

1. **Triage the 26 ability/item tags** into: covered-by-name (rewrite to read the artifact), genuinely-missing (wire it), or redundant-with-another-tag (delete one and say which).
2. **Decide the two-rulebook question.** Either `move-effects.json` is the source of truth for move behaviour and the 35 duplicated move tags should stop being derived, or the tags are and the engine should read them. Today both exist and only one is read, which is the worst of the three options.
3. **Build Tier 2** and downgrade every unproven LIVE.
4. **Then the registry**: a tag with no registered handler fails at load. That is the exhaustiveness check the architecture has been missing, and it is what stops this class rather than draining it.

`data/tag-consumption.json` ratchets **DEAD** — it may shrink and may never grow. `UNREACHED` and `STAGED` are properties of the sweep, recorded for information and never ratcheted, because ratcheting a number that measures the test rather than the engine is how a gate starts lying.